Métricas de software en los diferentes enfoques de gestión

1. Enfoque tradicional (procesos definidos)

El dominio de las métricas se divide en 3:

- **Proceso**: nivel organización. Las usa el ingeniero de procesos, son públicas y despersonalizadas (no importa qué proyecto o qué persona). Ej: desviación promedio entre esfuerzo estimado y real en toda la empresa.
- **Proyecto**: las usa el líder de proyecto, son privadas. Ej: costo estimado vs costo real, fechas reales vs programadas.
- **Producto**: las usa el equipo de desarrollo y testing. Ej: defectos por módulo, tamaño.

OJO: las métricas de proyecto se consolidan para armar las de proceso, que después son públicas para toda la organización.

Métricas básicas para cualquier proyecto (las 4 que hay que saber sí o sí):

1. Tamaño del producto (el qué)
2. Esfuerzo (el cuánto, en horas persona)
3. Tiempo calendario (el cuándo)
4. Defectos

No confundir tamaño con esfuerzo, ni esfuerzo con calendario. Ej: 40 horas de esfuerzo pueden ser 1 semana de calendario para 1 persona o 2 días si son 2 personas a full.

"El sueño del pibe": la filmina tiene la lista de todo lo que se podría medir según quién mide:

- Desarrollador: esfuerzo, duración estimada vs actual de una tarea, % de cobertura del unit test, defectos encontrados en unit test y en revisión por pares.
- Equipo: tamaño del producto, duración estimada y actual entre hitos, staffing, tareas planificadas vs completadas, distribución del esfuerzo, status y volatilidad de requerimientos, defectos en integración y prueba de sistema, defectos en peer reviews, % de tests ejecutados.
- Organización: tiempo calendario, performance planificada vs actual de esfuerzo y presupuesto, precisión de las estimaciones, defectos en release.

Es el "sueño" porque medir todo eso no es real, cuesta mucho. De ahí sale lo siguiente:

Manténgalo simple: "si estás a millas de tu destino no tiene sentido medir en milímetros".

Antes de agregar una métrica preguntarse:

- ¿Me da más información que la que ya tengo?
- ¿Tiene un beneficio práctico?
- ¿Me dice lo que quiero saber?

Si no cumple, no se mide. Además las métricas hay que balancearlas entre calidad, alcance, restricciones, necesidades/expectativas y proceso.

2. Enfoque ágil (procesos empíricos)

Regla de oro: la medición es una salida, no una actividad. No hay tareas para "tomar métricas", salen solas del trabajo. Filosofía minimalista: medir lo que sea necesario y nada más. Dos principios del manifiesto que guían qué medir:

- "Nuestra mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso, funcionando."
- "El software funcionando es la principal medida de progreso."

Por eso se mide el producto y no el proceso. Tampoco se compara entre equipos ni se usan para evaluar personas.

Velocidad

- Suma de story points de las US terminadas y aceptadas por el PO en el sprint.
- Se CALCULA al final (en la review), no se estima.
- Si una US quedó a medias o el PO la rechazó suma 0.
- Mirar el promedio, no un sprint suelto (oscila).
- Ej: comprometieron 8 + 5 + 3, terminaron la de 8 y la de 5, la de 3 quedó a medias → velocidad = 13.
- Sirve para proyectar: SP que quedan en el backlog / velocidad promedio = sprints que faltan.

Capacidad

- Es una ESTIMACIÓN de cuánto puede comprometer el equipo en el próximo sprint, se hace en la planning.
- Capacidad = horas disponibles por día x días disponibles del sprint.
- El ejemplo de la filmina: sprints 1 y 2 con 240 h, el 3 y 4 con 480 h, 5 y 6 con 420 h y el 7 con 480 h (total 2760 h, 335 SP). Va cambiando porque la gente viaja, se toma licencia, etc.
- Equipos maduros la estiman en SP; los nuevos, en horas ideales.

Running Tested Features (RTF)

- Cantidad de features funcionando y testeadas.
- Problema: no tiene en cuenta el tamaño de cada feature (una chica y una enorme cuentan igual), por eso casi no se usa.

Resumen ágil: velocidad, capacidad, RTF.

3. Kanban (procesos empíricos con enfoque Lean)

Acá el foco es el flujo, no hay sprints. Se mide cuánto tarda cada ítem en recorrer el tablero.

- **Lead time** = vista del CLIENTE. Desde que se pide el ítem hasta que se entrega. Es el ritmo de entrega. Se mide en días de trabajo.
- **Cycle time** = vista INTERNA. Desde que se empieza a trabajar hasta que se termina. Es el ritmo de terminación, una medición más mecánica de la capacidad del proceso. Se mide en días de trabajo o esfuerzo.
- **Touch time**: el tiempo en que el ítem realmente fue trabajado ("tocado"). Cuenta los días en columnas de trabajo en curso, no los de colas/buffer ni los bloqueados.
- **Eficiencia del ciclo de proceso** = touch time / elapsed time (lead time). Cuanto más cerca de 1, menos espera.

Ej que hice para entenderlo: un pedido entra el día 1, se empieza el día 11, se entrega el día 15 y estuvo 1 día parado esperando testing.

Lead = 14 días · Cycle = 4 días · Touch = 3 días · Eficiencia = 3/14 ≈ 0,21. O sea que casi el 80% del tiempo estuvo esperando.

Métricas orientadas a servicio (también de Kanban):

- Expectativa de nivel de servicio: lo que el cliente espera.
- Capacidad del nivel de servicio: lo que el sistema puede entregar.

- Acuerdo de nivel de servicio: lo que se acuerda con el cliente.
- Umbral de adecuación del servicio: el nivel por debajo del cual ya es inaceptable para el cliente.

4. Resumen de los 3 enfoques

- Tradicionales: esfuerzo, tiempo, costos, defectos.
- Ágiles: velocidad, capacidad, running tested features.
- Lean: lead time, cycle time, touch time, eficiencia del proceso.

5. ¿Y para el producto qué se mide?

Sirve en cualquier enfoque:

- **Tamaño**: casos de uso, líneas de código, features, requerimientos, clases, puntos de historia.
- **Defectos**: defectos por severidad, densidad de defectos (defectos / tamaño).
- **Cobertura** de pruebas.

.